

Lec 5) Binary Search Tree

BST

Special kind of binary tree
It is efficient structure to organize data for
quick search as well as quick update

What data structure will you use
to store modifiable collection ?

* Search

* insert

* delete

	Search	Access	Modify	Insert	Delete
Array	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(n)$	$O(n)$	$O(n)$
BST	$O(\log_2 n)$				

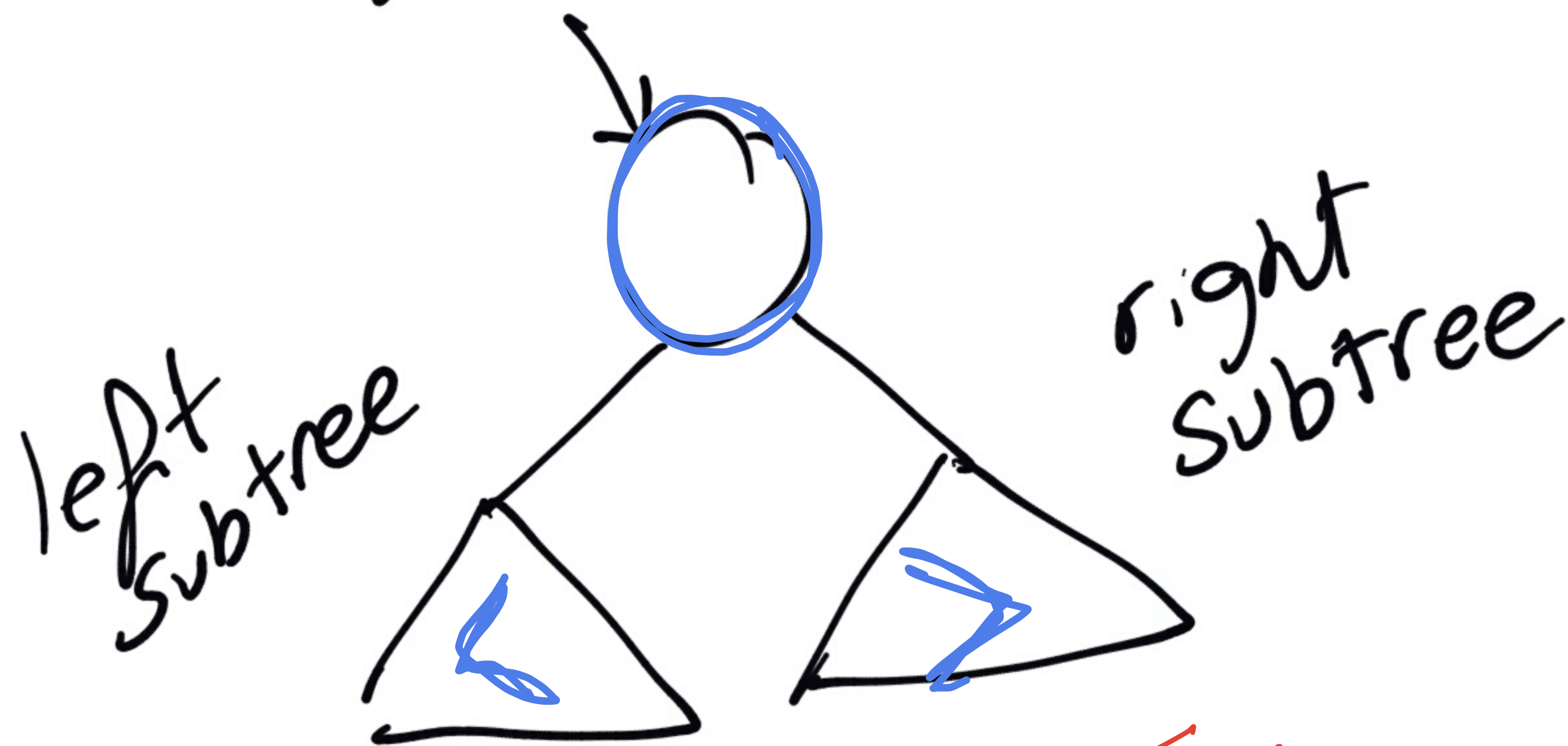
Recall

$O(\log_2 n)$ is much better
than $O(n)$

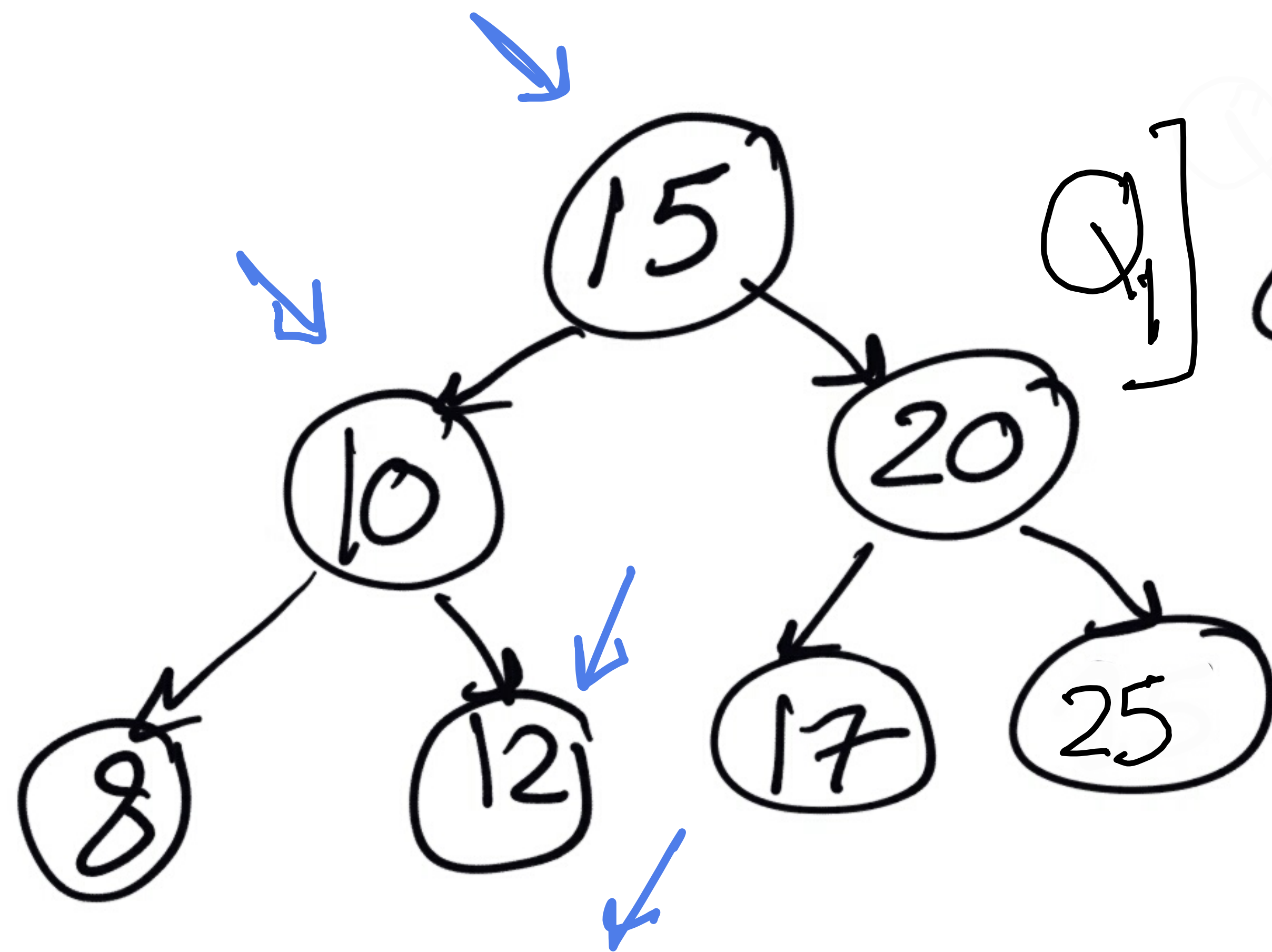
BST for each node values of all nodes
in left subtree is lesser, and values of
all node in right subtree is greater

Min element
= left most
node

Max-element
= right most node



[Assume no duplication]



Q1] Can we replace 16 by 12 ?
 NO

Q2] How can we search for 12 ?

$10 < 12 < 15$
 $15 < 17 < 20$

Complexity

$O(h)$

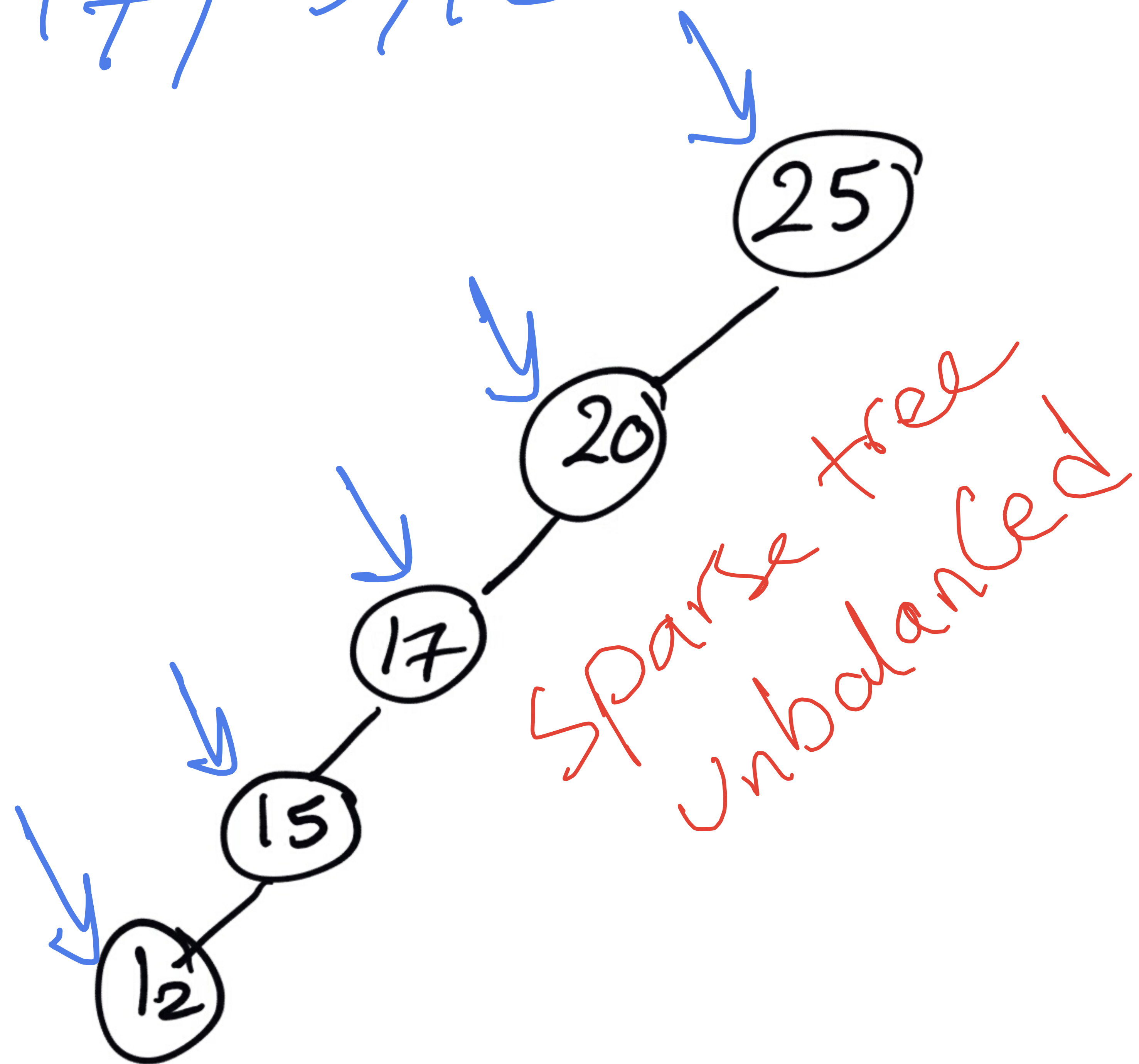
CBST

$O(\log_2 n)$

insert 25, 20, 17, 15, 12

Complexity $O(N)$

Worst Case



BST operations

1 Search

2 insert

3 delete

Best Case $O(\log_2 n)$
average case $O(\log_2 n)$

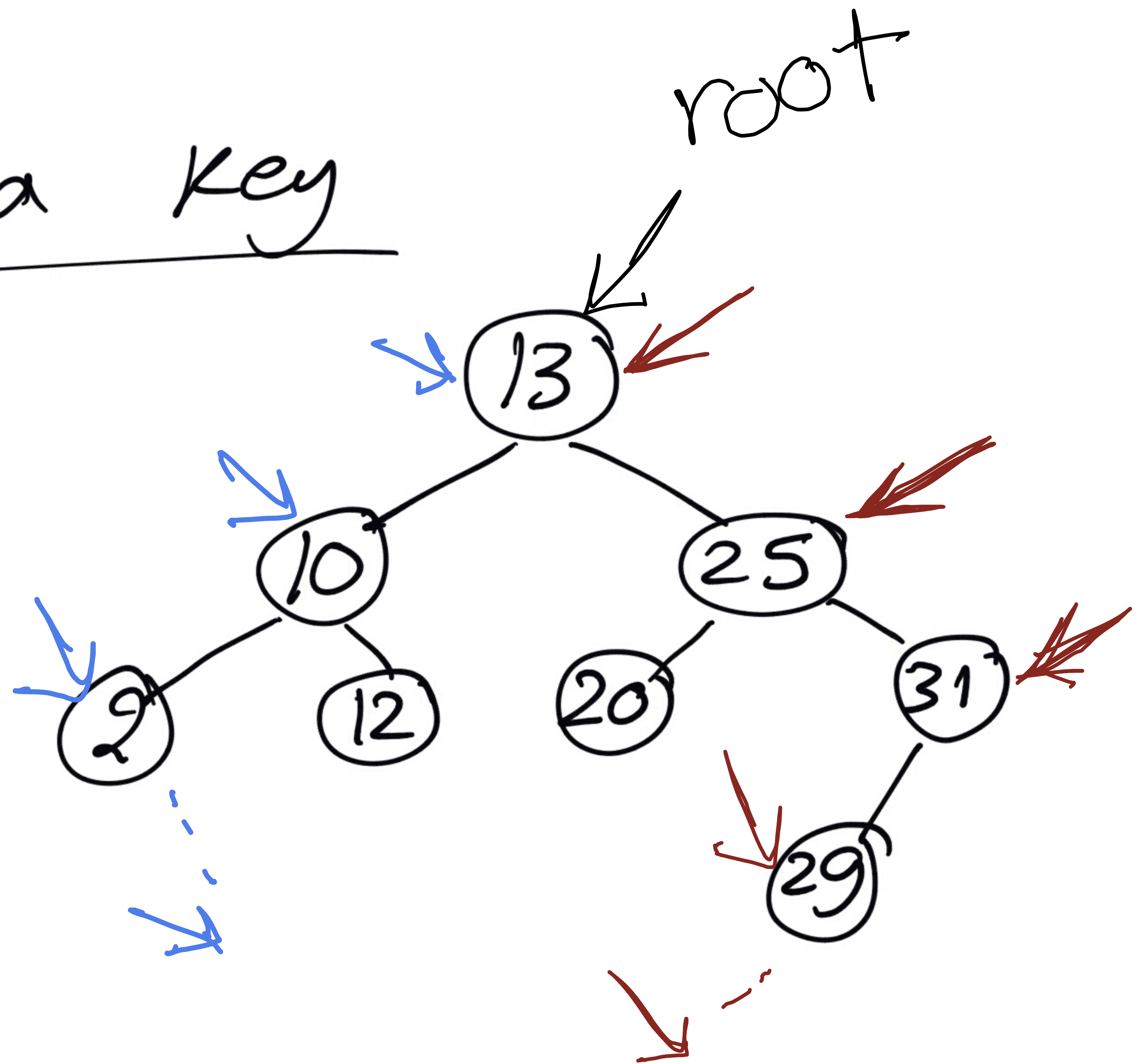
Worst Case
 $O(n)$

1 Search for a Key

Search for 3
Not Found

Search for 28

Not Found



Boolean Search (root, Key)

Recursive

```
{ if (root = NULL) return F;
```

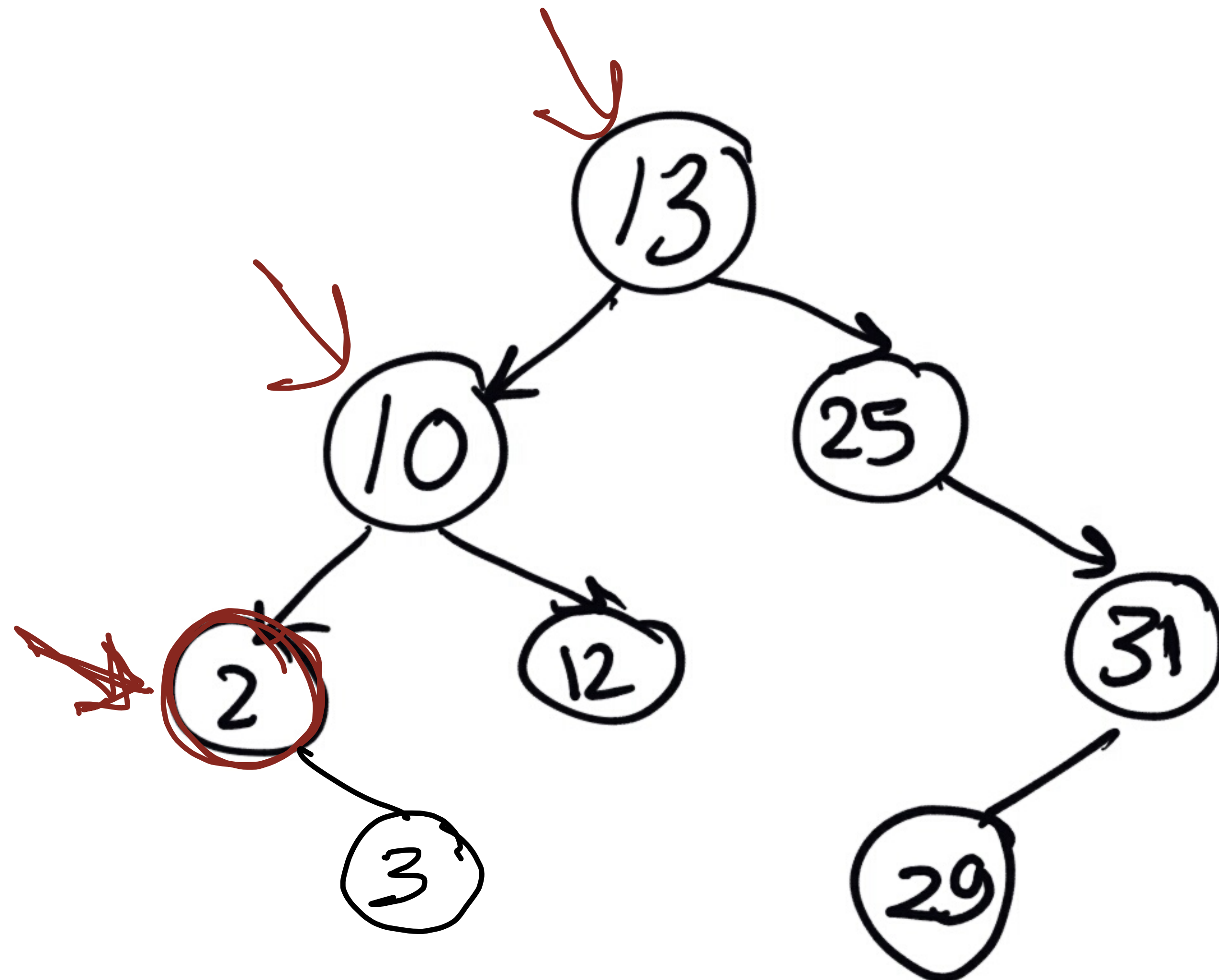
```
else if (root.data = Key) return T;
```

```
else if (Key < root.data) Search(root->left, Key)
```

```
else Search(root->right, Key)
```

```
}
```


Problem #1 Write a recursive function to
find the minimum element in BST




```
int findMin(root) error
```

```
{ if ( // empty tree  
      root = Null ) return -1 ;
```

// left most node

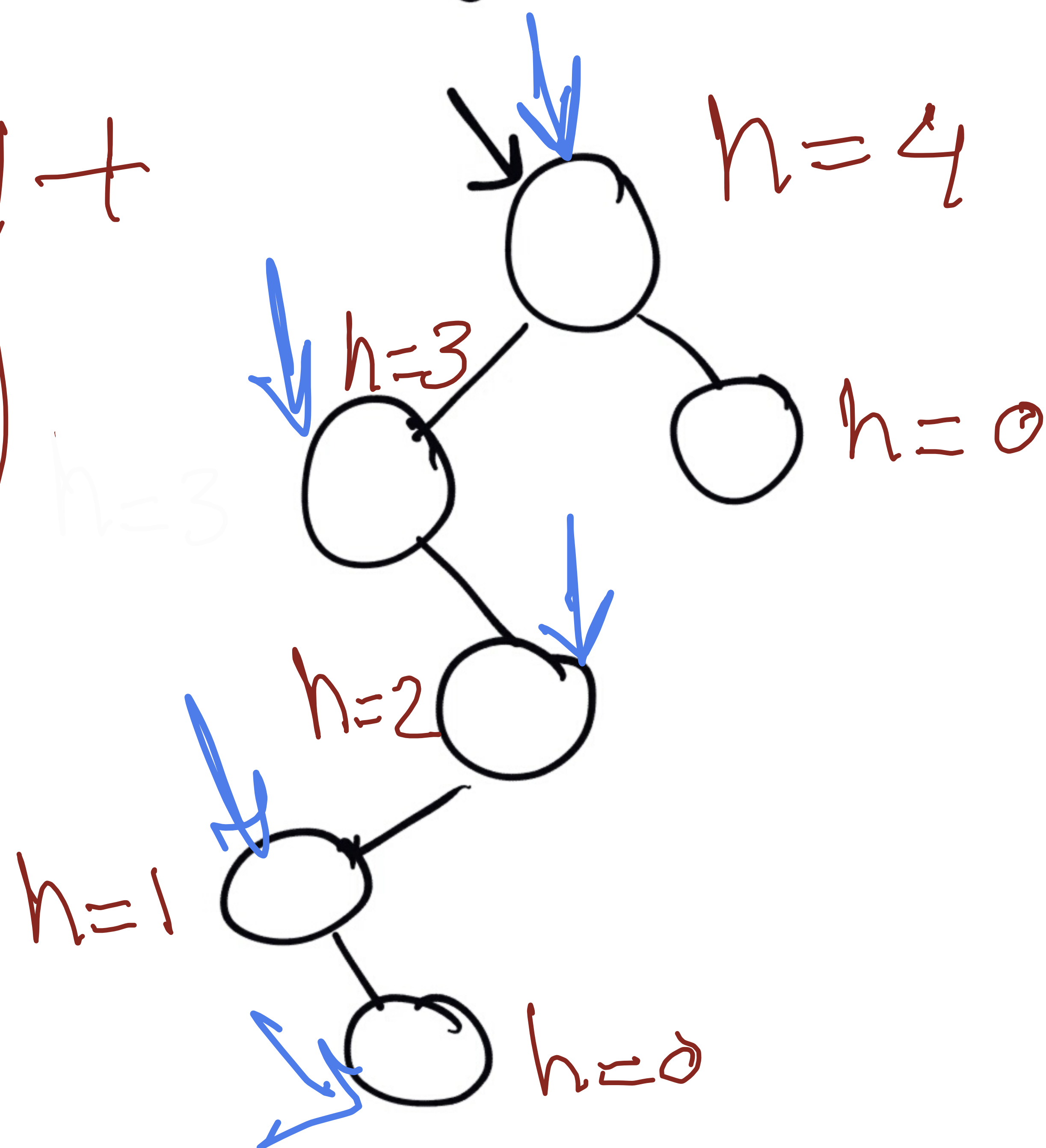
```
else if ( root -> left = Null ) return root.data
```

```
return findMin( root -> left ) ;
```

```
}
```


Problem #2 Write a recursive function to
Compute the height of BST

height of tree = 1 +
 $\max(\text{height left subtree}, \text{height right subtree})$




```
int findHeight(root)
    // leaf node
    { if (root->left==Null) && (root->right==Null) return 0
```

```
    return 1 + Max(findHeight(root->left),
                    findHeight(root->right))
```

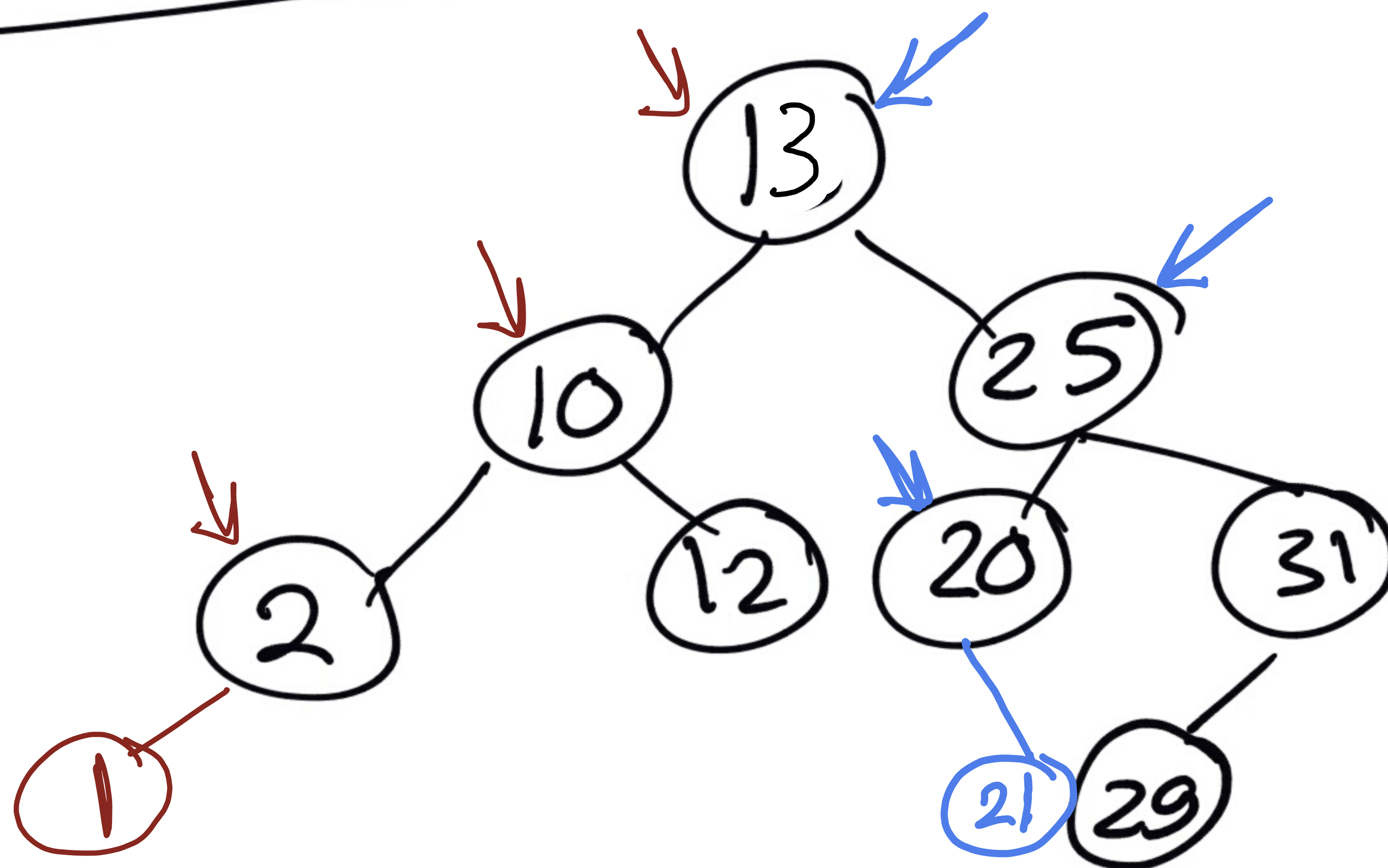
}

2 Insert node in BST

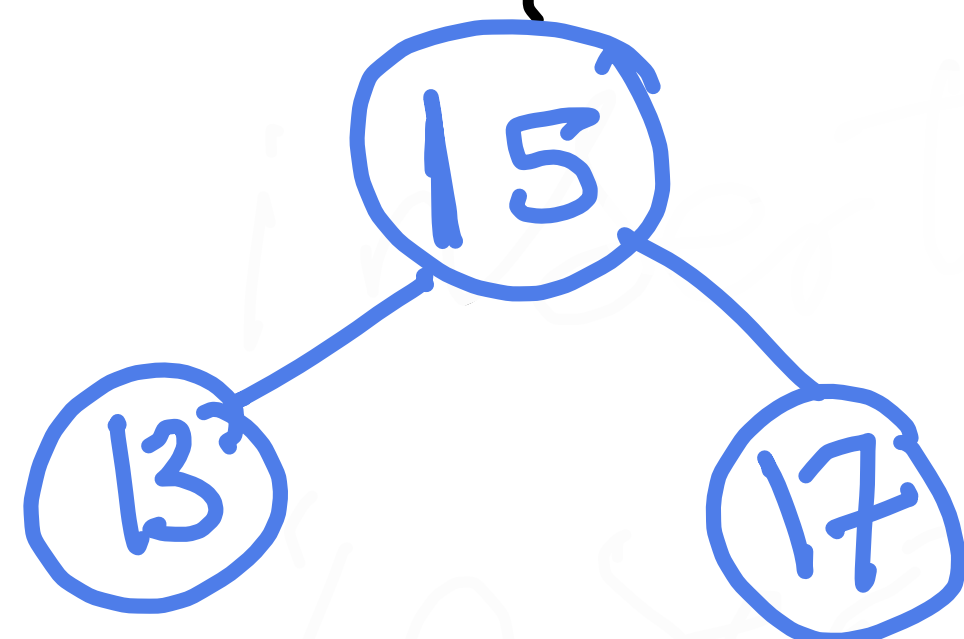
leaf node

insert ①

insert ②

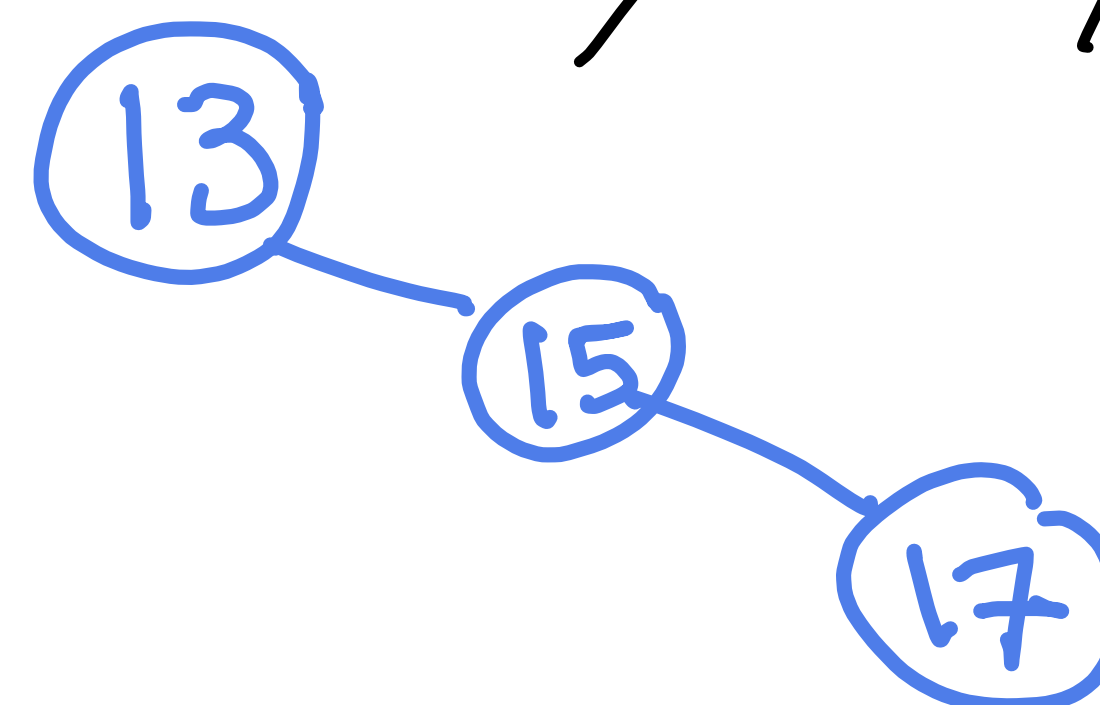


insert 15, 13, 17

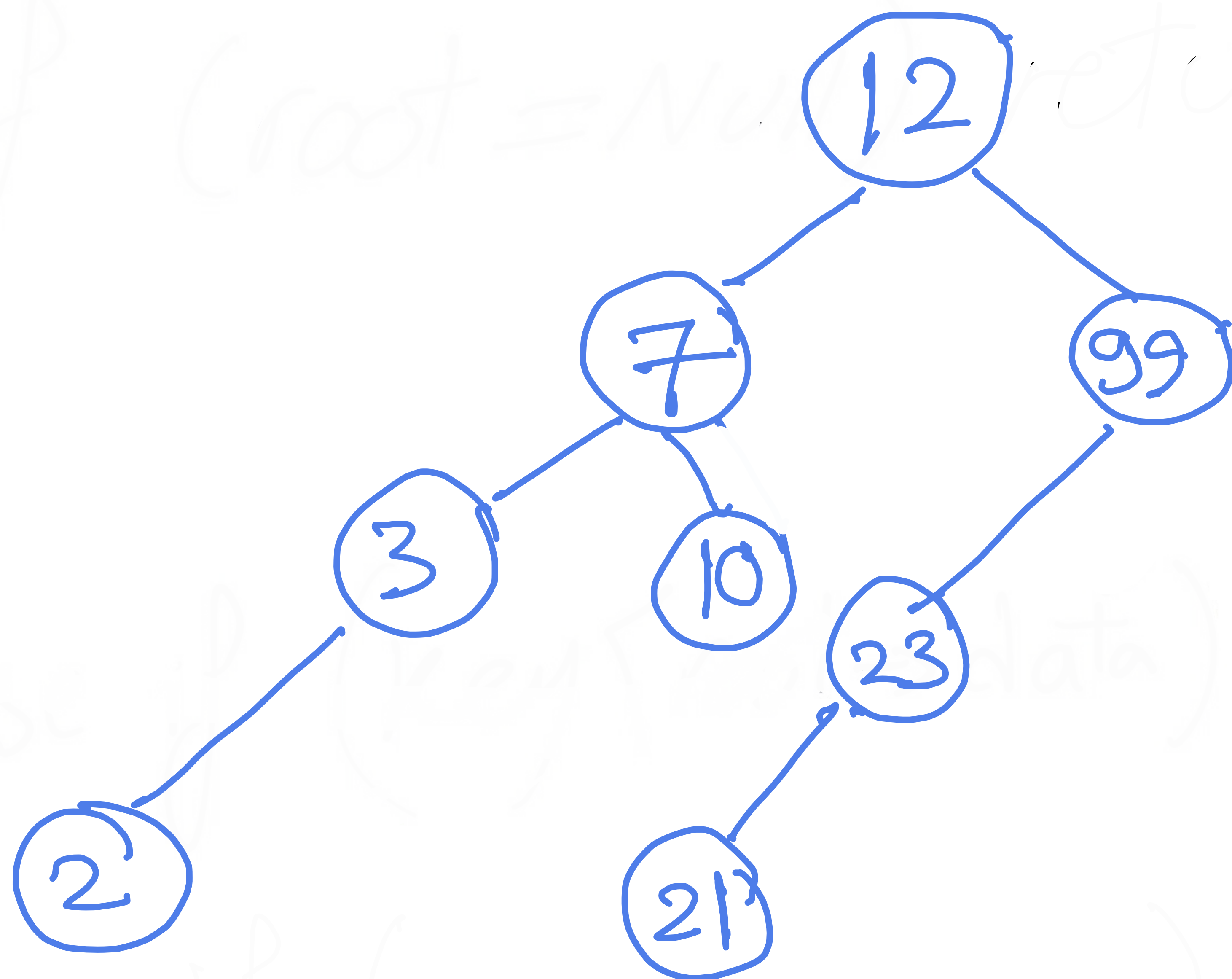


insertion
order

insert 13, 15, 17



insert 12, 7, 3, 99, 23, 21, 10, 2

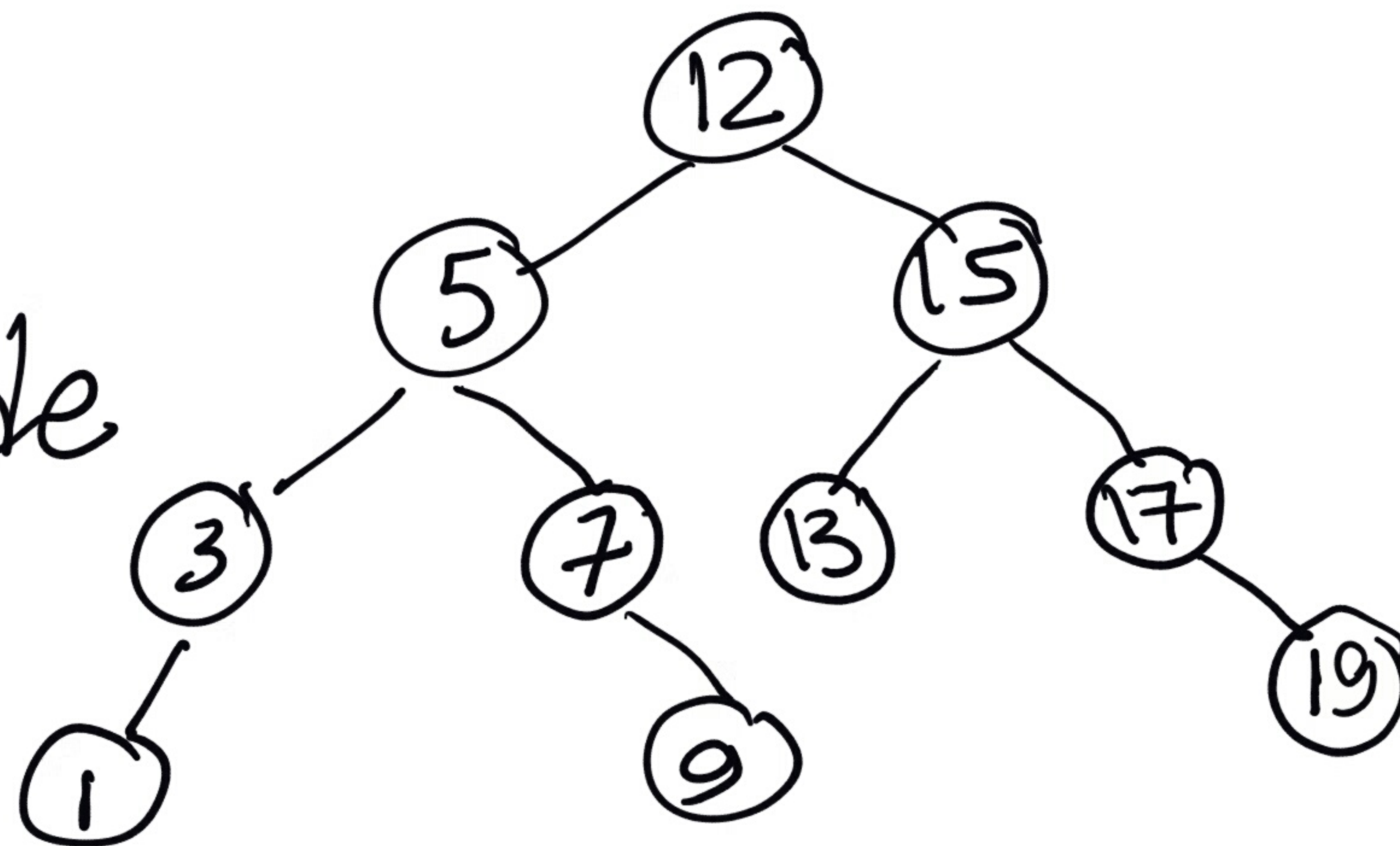


3 Delete a node

Case 1

leaf node

e.g. delete 9



Case 2 one child

e.g. delete 17

link its parent to its child

Case 3

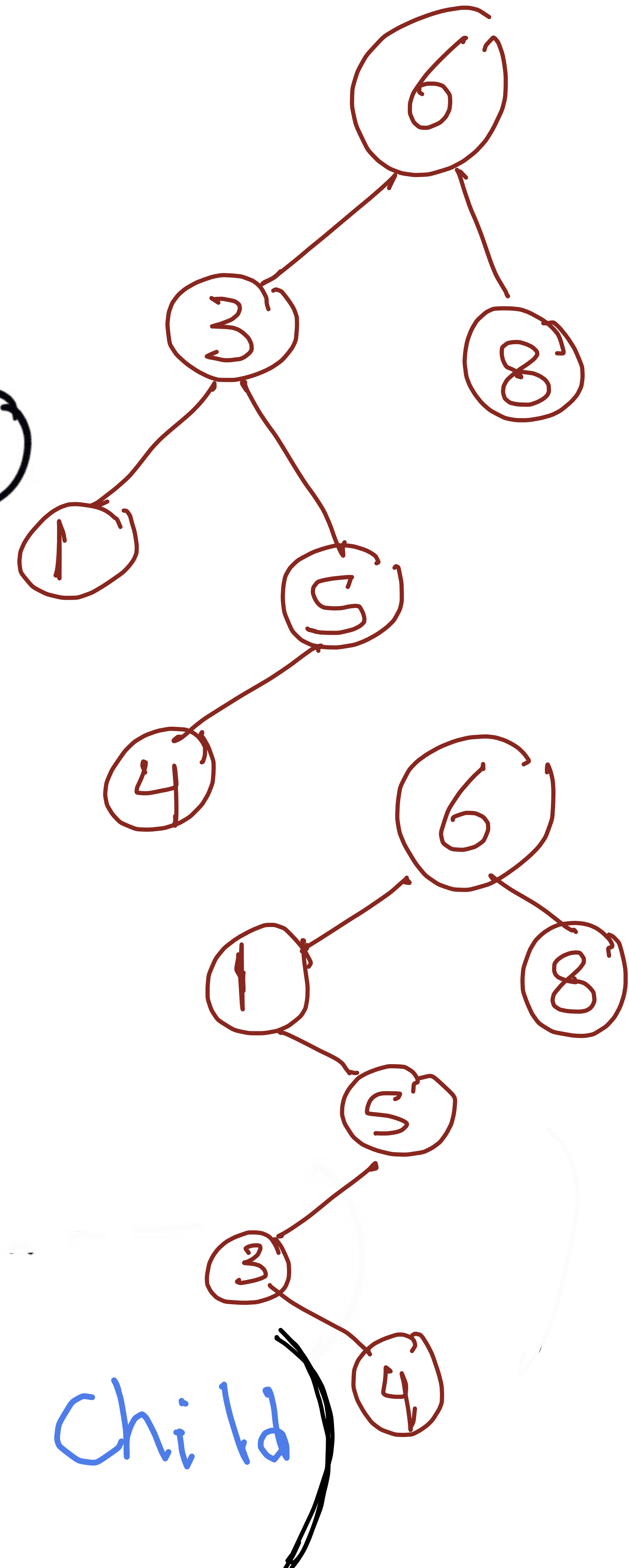
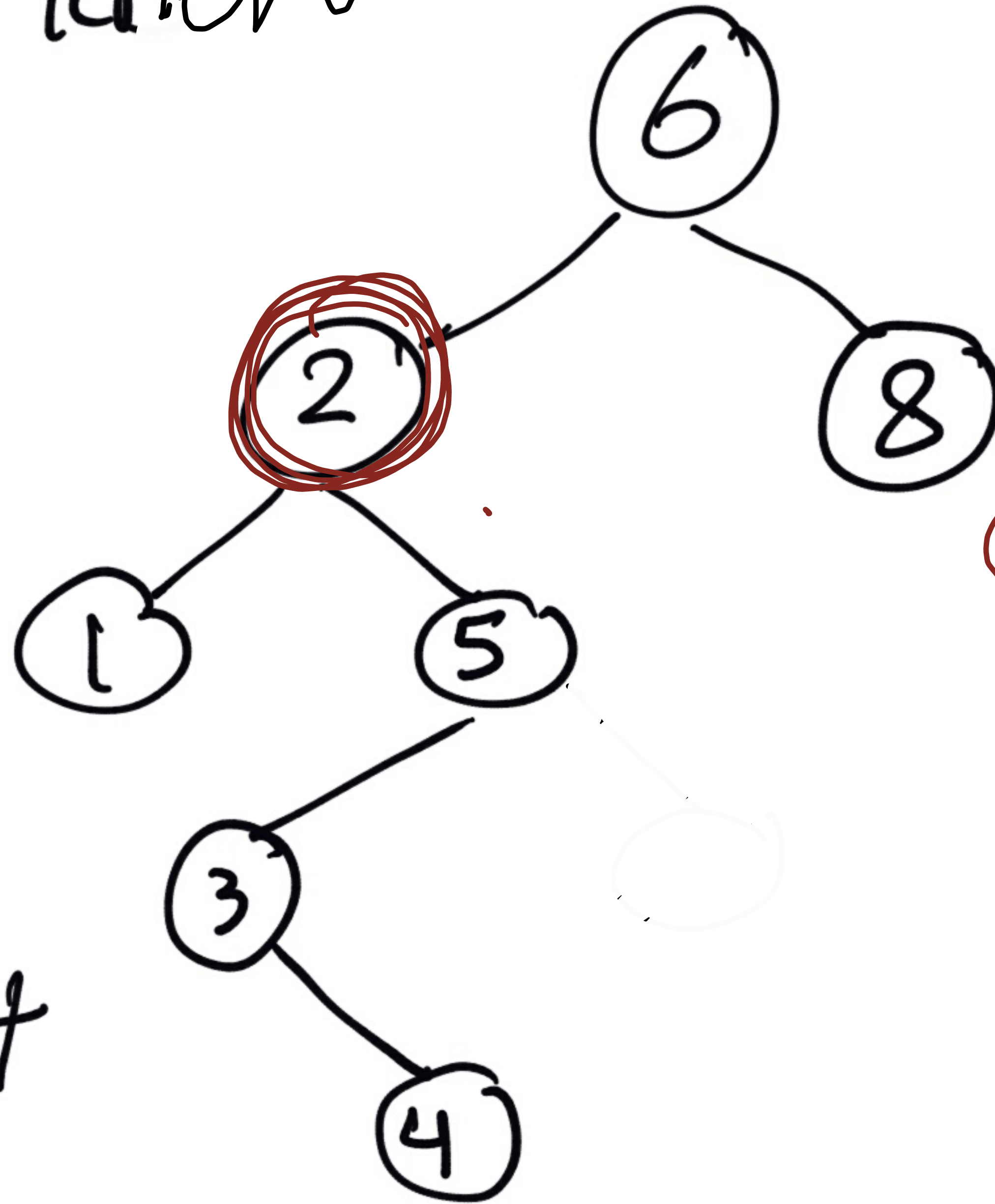
two children

delete 2

① Search for Min. element
in right subtree

② Copy the value in target
node

③ delete the min element (it may has
one or zero child)



Conclusion

BST

Search
Insertion
Deletion

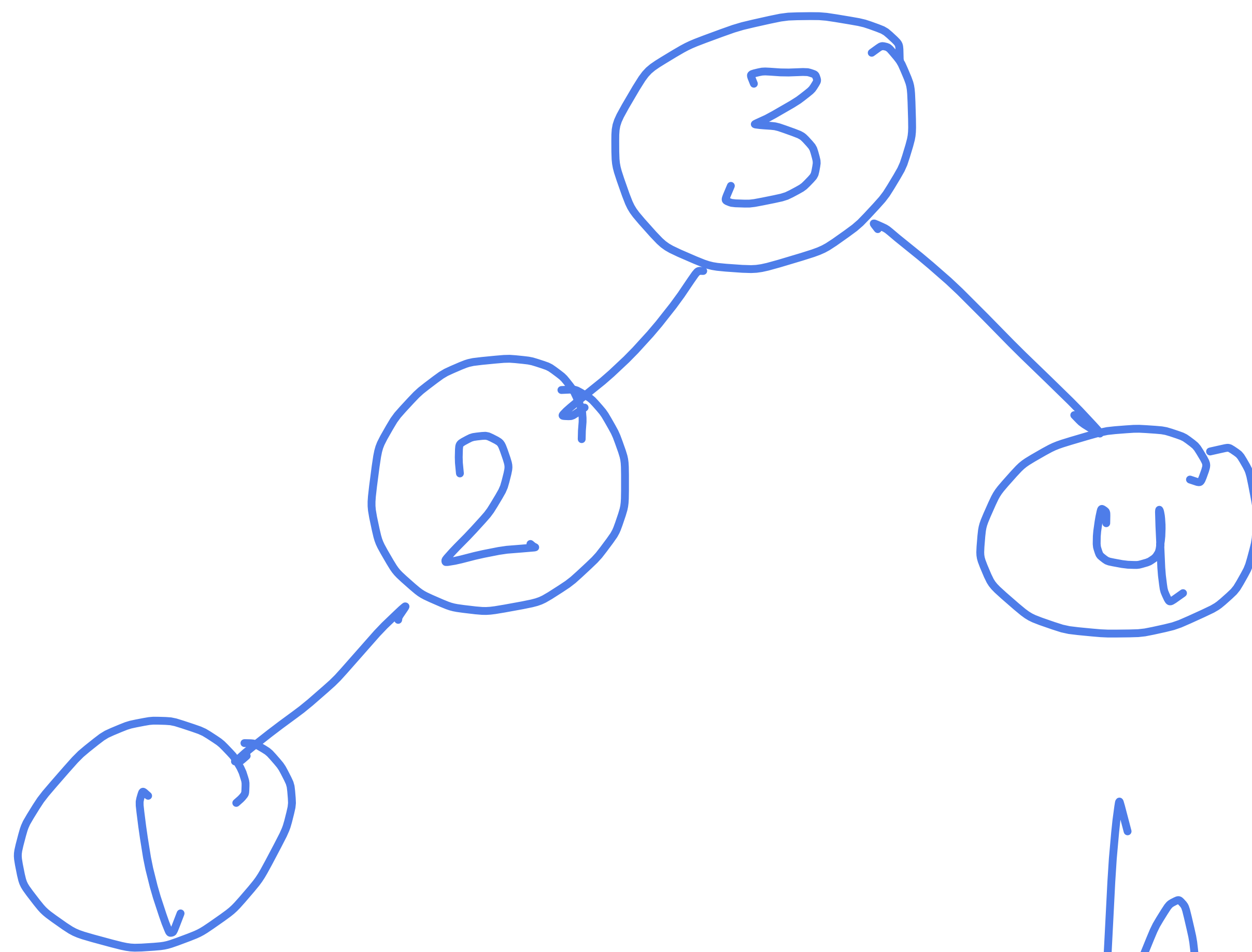
$O(\log_2 n)$

on the
average

What are BST draw backs

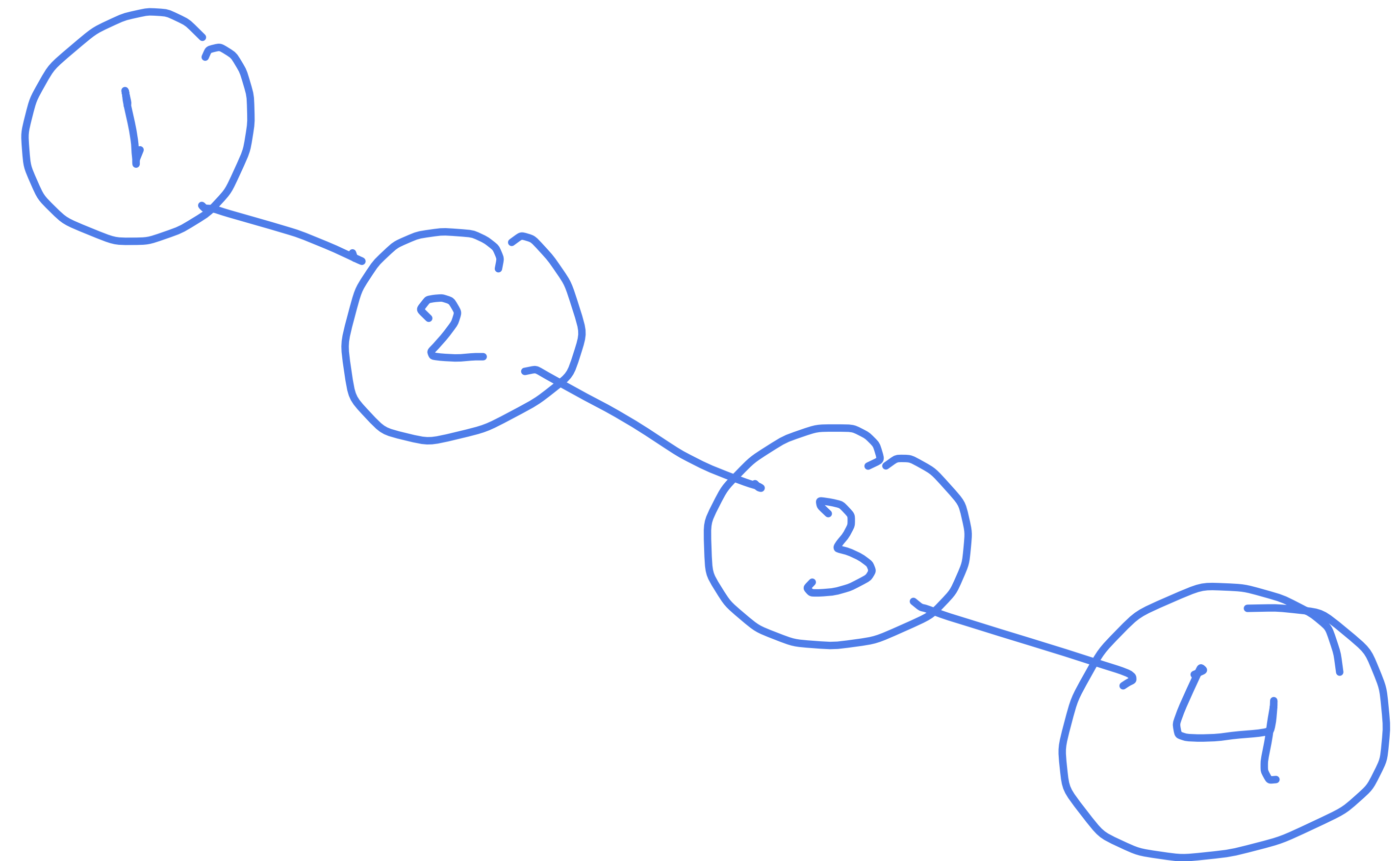
height is not under control
it depends on the insertion
order of element

insert 3, 2, 1, 4



$$h = 2$$

insert 1, 2, 3, 4



$$h = 3$$

What should we do to Control
the height $\Rightarrow$

A V L Tree
Adelson-Velsky and Landis

Balance Factor BF of a node

$BF = \text{height of left subtree} - \text{height of right subtree}$

AVL tree is a special kind of BST

that is used to maintain height
BF for all nodes = $\{-1, 0, 1\}$

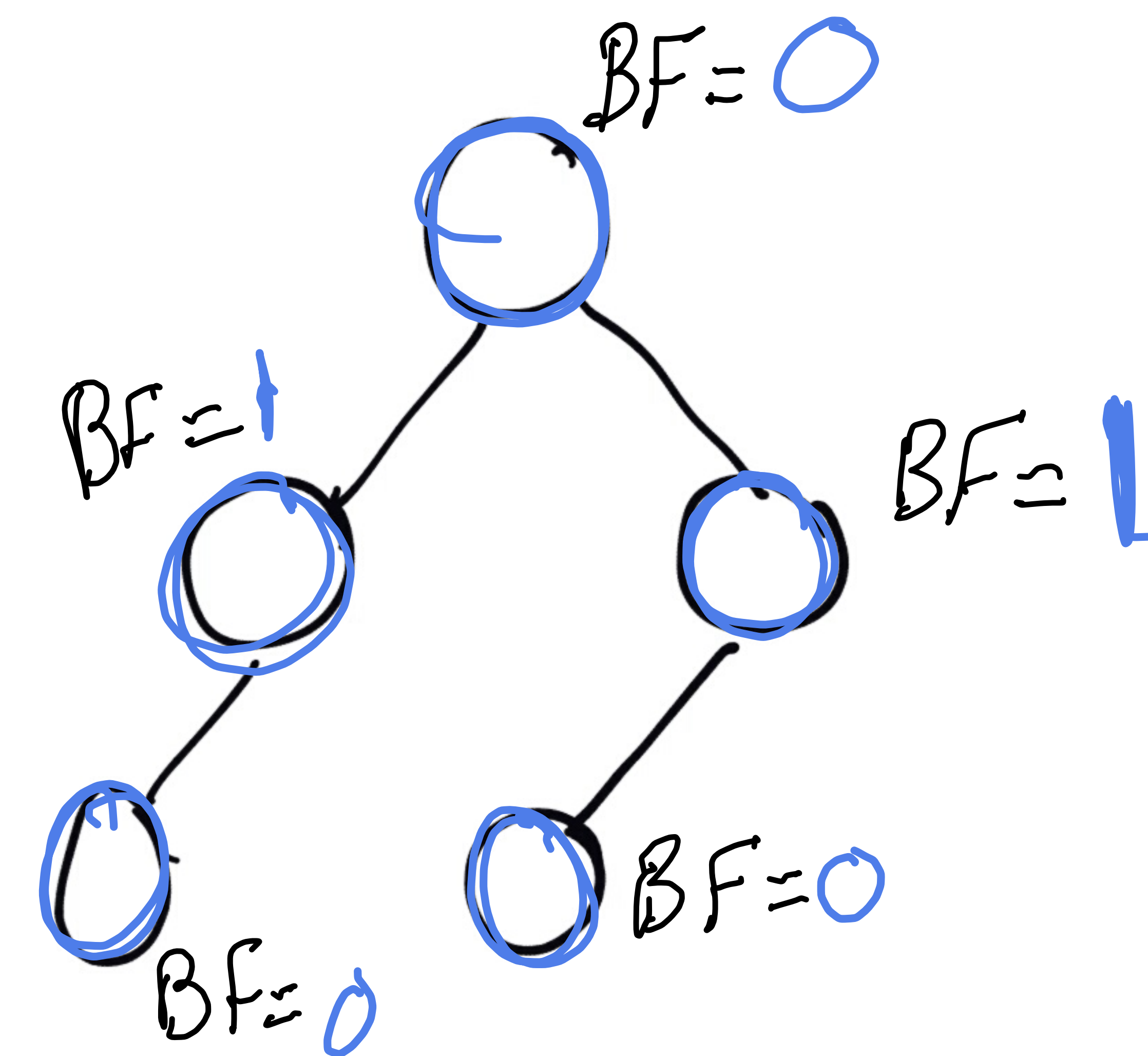
AVL Tree = Balanced BST

is a BST in which the absolute difference
between height of left subtree and
right subtree of any node does not
exceed 1

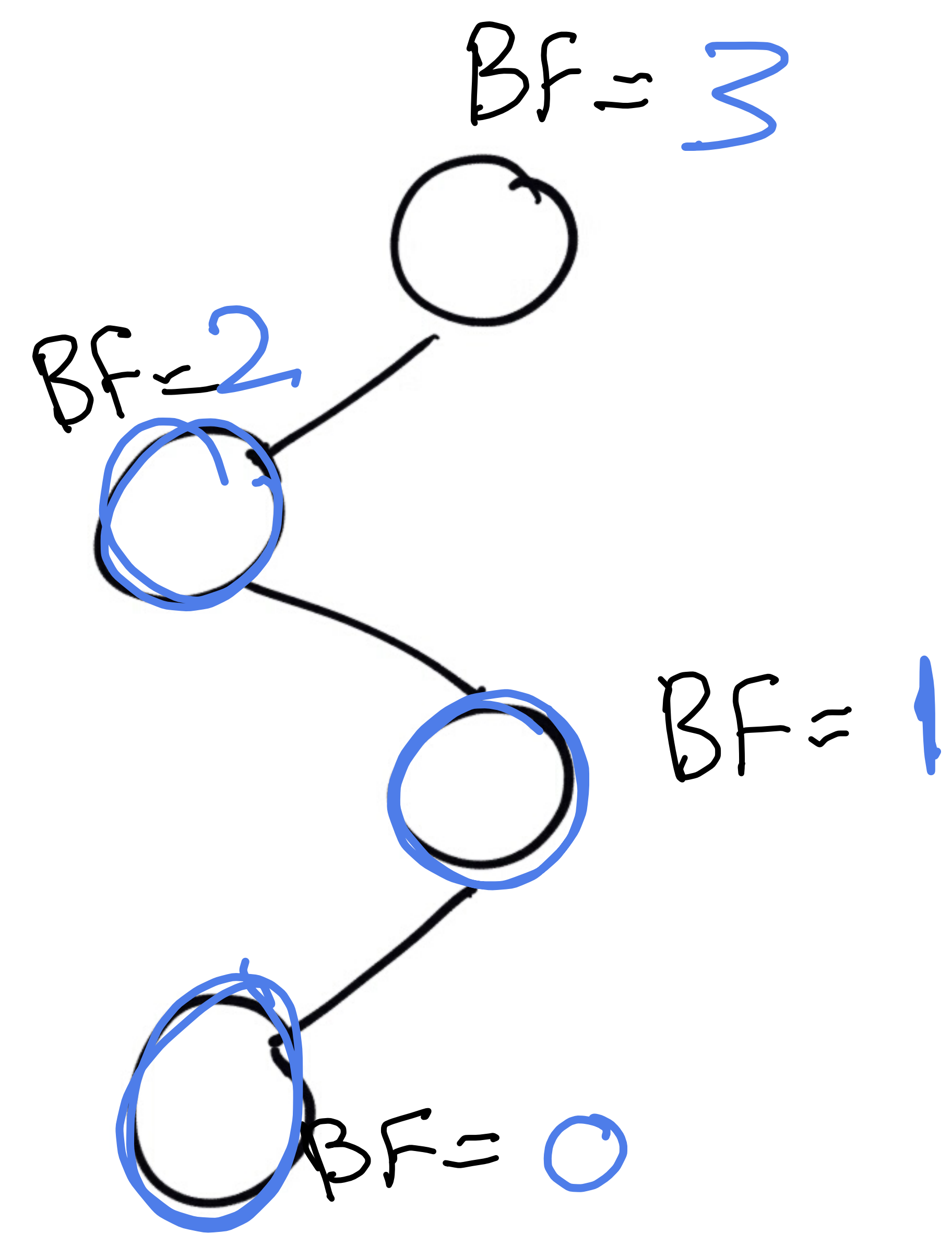
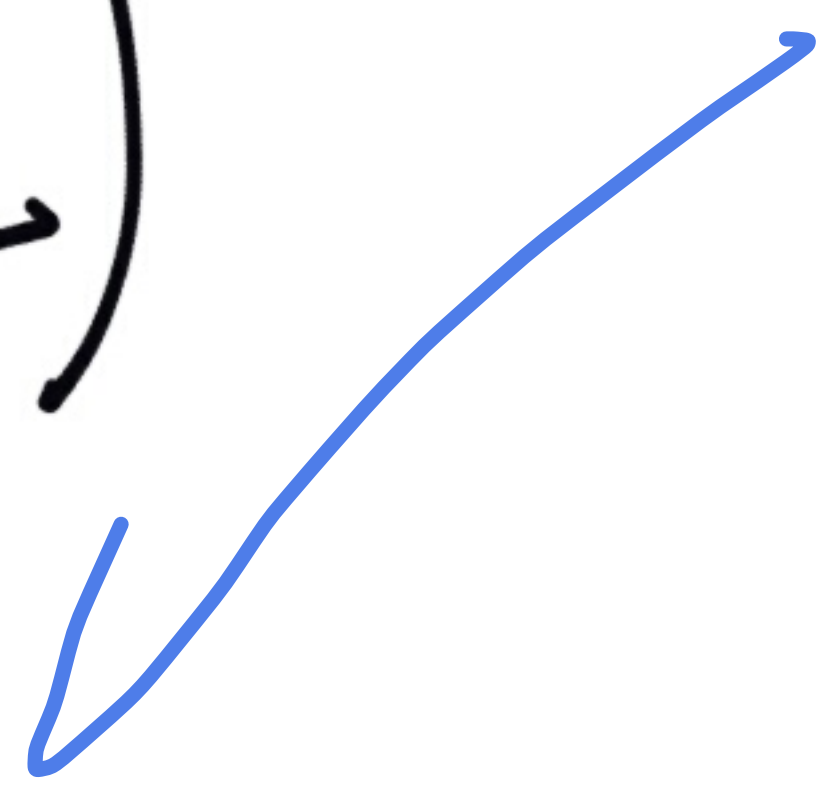
$$|BF| < 1$$

$$-1 < BF < 1$$

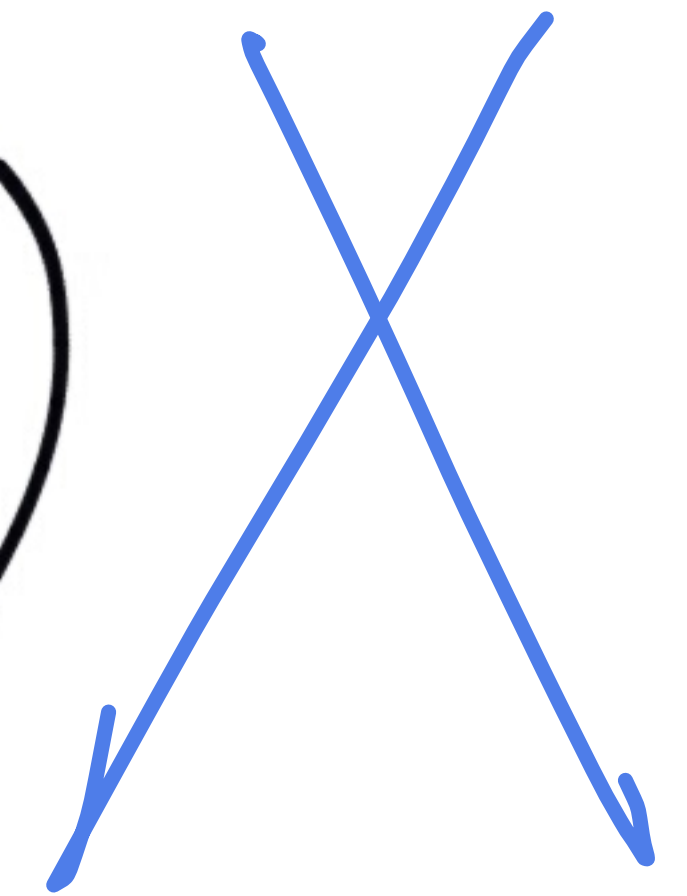
$$[-1, 0, 1]$$



balanced(AVL)



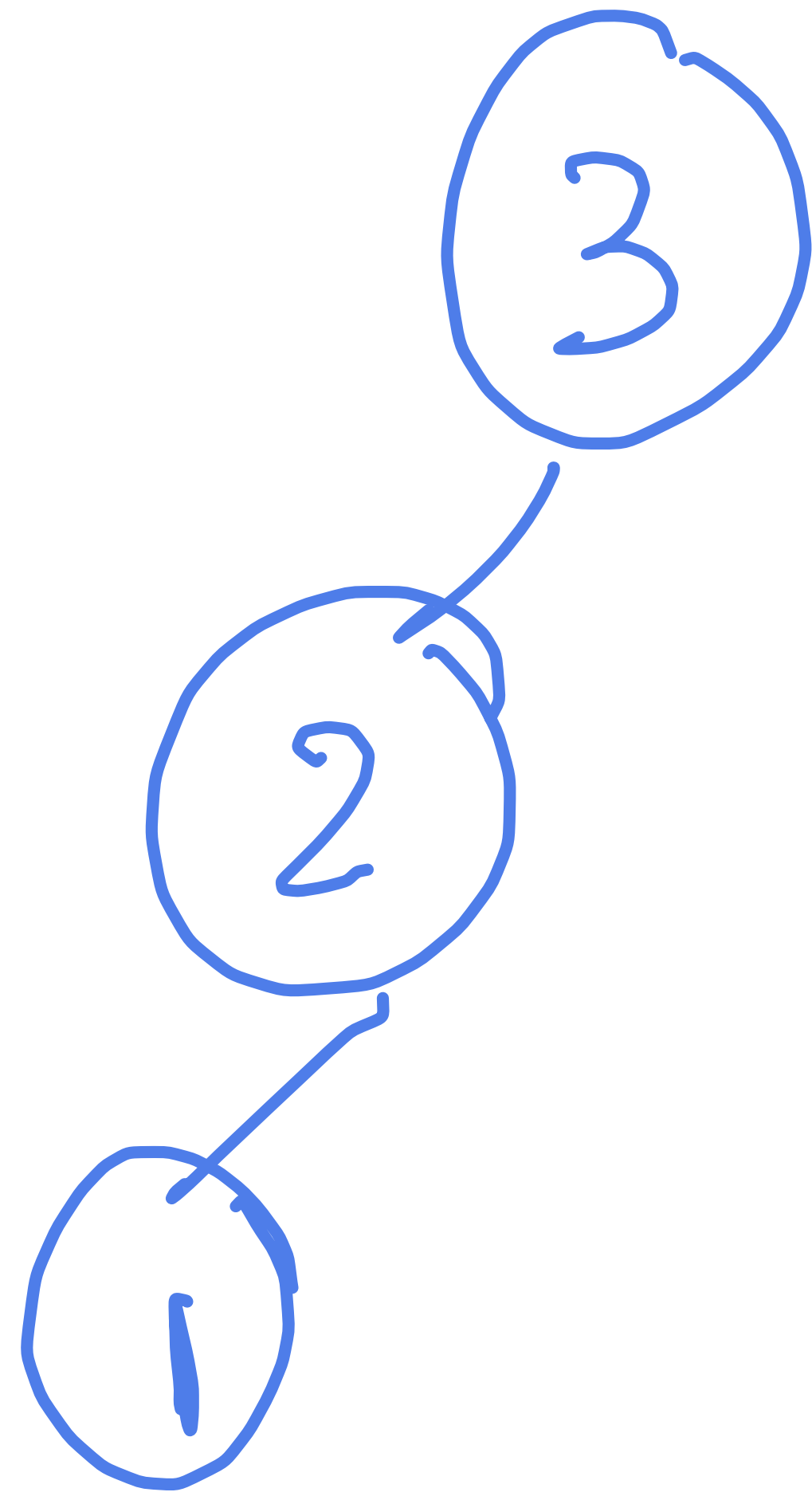
balanced(AVL)



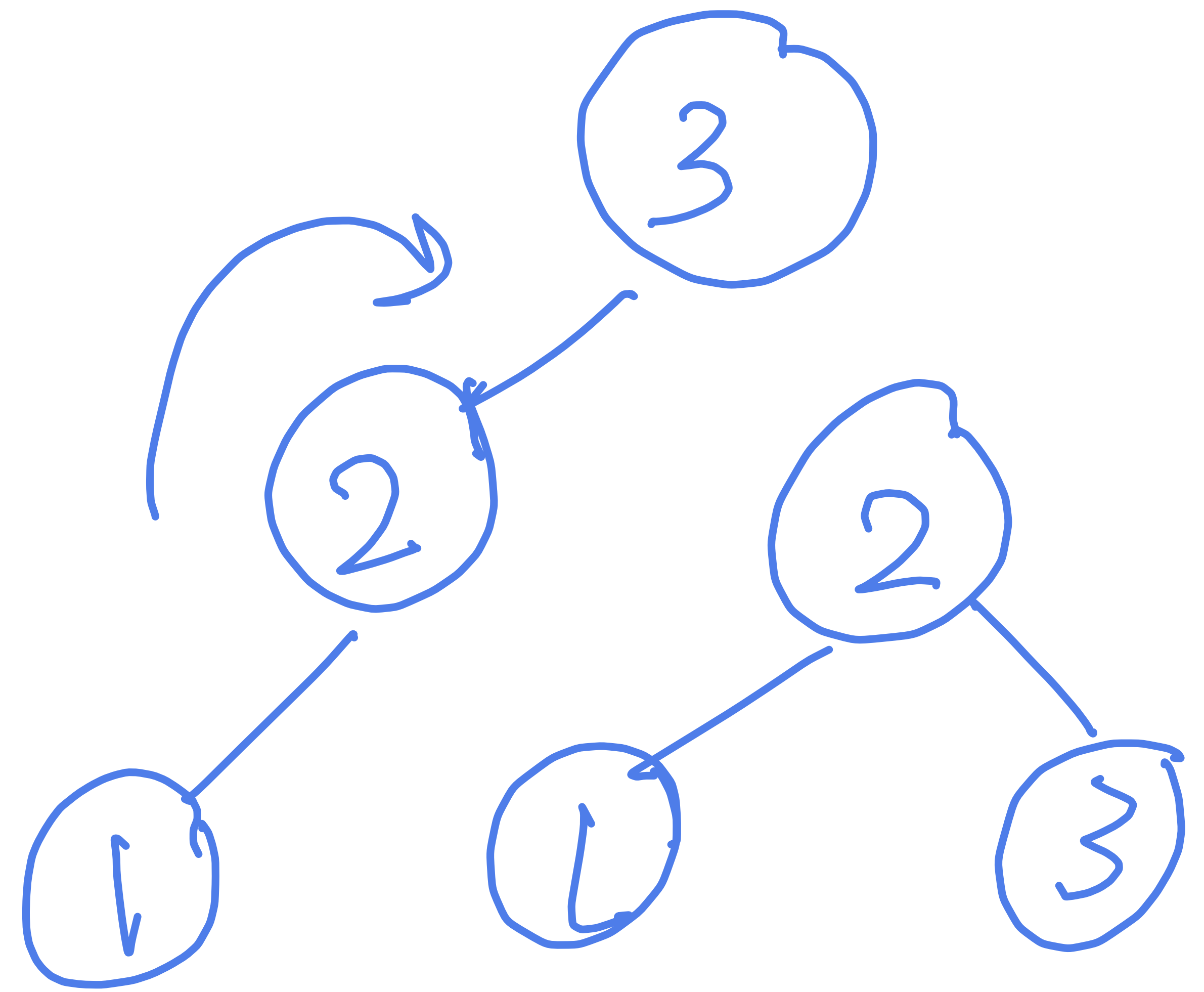
insert ③

insert ②

insert ①



BST



AVL

The only solution to maintain BF for
AVL Tree while insertion is rotation

AVL Tree

Insertion techniques

L-L / R-R
L→R R-L

Balancing
rotation
Strategies

Red_black tree

(same as AVL tree with
less rotation)